

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине «Машинное обучение»

на тему «Библиотеки Python для машинного обучения. Работа с данными»

Выполнил:

Студент _Солодкий М.И. __

Группа ___ИПБП41_____

Проверил:

Дольников Максим Генадьевич

Ростов-на-Дону

2026 г.

Цель работы:

Изучить основные библиотеки Python для машинного обучения (NumPy, Pandas, Matplotlib, SciPy, scikit-learn, TensorFlow, Keras, PyTorch) и приобрести практические навыки работы с табличными данными: чтение/запись, предобработка, статистический анализ, тензорные вычисления, обработка изображений.

Задачи:

1. Освоить чтение и запись данных в различных форматах.
2. Научиться выявлять и обрабатывать проблемы в данных.
3. Реализовать собственные функции масштабирования и сравнить их со sklearn.
4. Освоить статистический анализ данных средствами NumPy, Pandas, SciPy.
5. Научиться работать с тензорами TensorFlow, PyTorch, Keras.
6. Освоить базовую обработку изображений средствами SciPy.

Выполнение работы:

Раздел 1

Вариант №14

Задание 1. Чтение файлов.

Реализована универсальная функция чтения табличных данных, которая по расширению файла выбирает подходящий способ загрузки: для csv используется pandas.read_csv, для xlsx — pandas.read_excel, для txt — pandas.read_csv с разделителем из пробелов, для mat — scipy.io.loadmat с автоматическим выбором первого двумерного массива. Датасет dataset.csv

содержит 500 строк и 7 столбцов: id, timestamp, category, x1, x2, x4, y. Данные загружены успешно, структура корректная.

1	id,timestamp,category,x1,x2,x4,y
2	1,2024-01-01 00:00:00,A,0.6835119515543189,0.17517845559778916,-0.8096035199216461,38.30243095992137
3	2,2024-01-01 01:00:00,A,3.7523416784317725,0.2022088913599509,0.4250610444640897,181.42270213903245
4	3,2024-01-01 02:00:00,B,1.9008476763721005,0.7691980363788655,-0.4698393470709161,132.61413327478337
5	4,2024-01-01 03:00:00,A,-1.1538073113248062,1.1543328354359643,-0.005452270878279103,-16.476769933677367
6	5,2024-01-01 04:00:00,C,-1.796829342696716,1.2278867700250227,0.5622842480036996,-76.58910720230492
7	6,2024-01-01 05:00:00,B,0.9838383430130114,1.2159268897025404,0.03142226981093782,63.58725856668329
8	7,2024-01-01 06:00:00,A,-2.6404664140412843,1.3894581524202798,-0.4003858905265686,-61.28433639916948
9	8,2024-01-01 07:00:00,C,3.6629175317087075,1.3609807106267675,-0.06060980392583083,199.6449210190394
10	9,2024-01-01 08:00:00,A,2.358880241442574,1.3197207580220327,-0.02436295920533532,142.26211751007563
11	10,2024-01-01 09:00:00,A,-0.9383513042094096,0.9727677591046489,-0.2890110298821277,-3.7302428912153047
12	11,2024-01-01 10:00:00,A,-3.426269058181755,0.6047560323050803,-0.15879606266710236,-128.45897746791246
13	12,2024-01-01 11:00:00,A,2.7077447483308257,0.6024408263963378,1.719647170504717,119.12886533076065
14	13,2024-01-01 12:00:00,A,-0.22907969050523577,0.8681917902100307,0.7049194479412387,0.004412668841205392
15	14,2024-01-01 13:00:00,A,2.4756326239469235,0.2625353374085602,-0.12648031802916035,116.52441997340075

Рисунок 1 - Датасет

Задание 2. Запись файлов.

Реализована функция, сохраняющая таблицу сразу во все поддерживаемые форматы (xlsx, csv, txt, mat). К имени каждого файла добавляется метка времени, чтобы результаты разных запусков не перезаписывали друг друга.

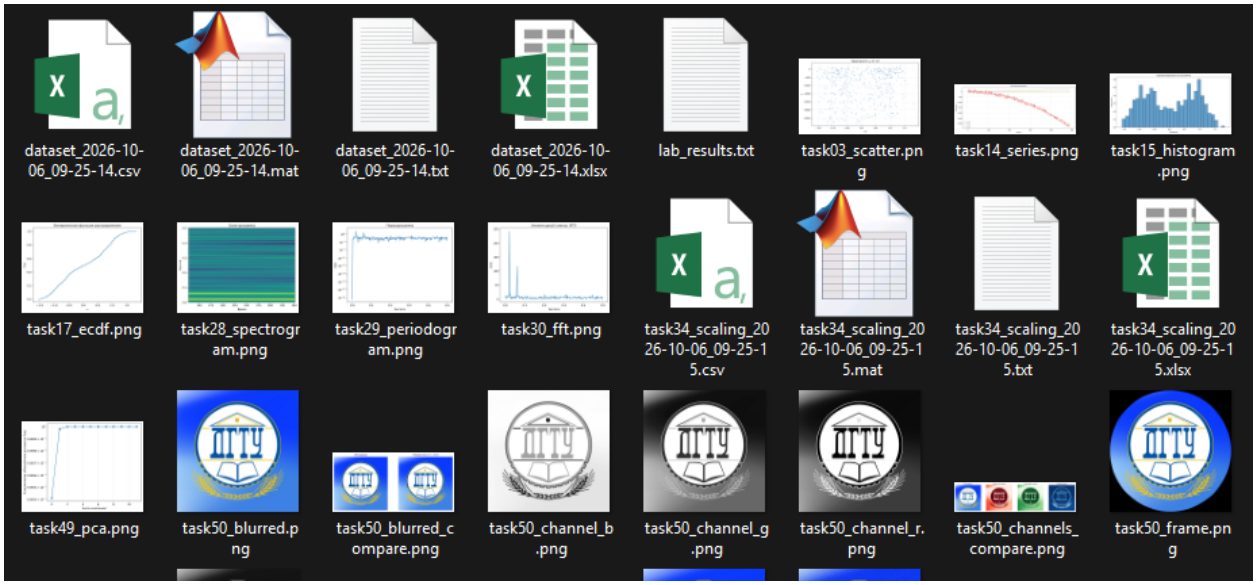


Рисунок 2 – Сохранившиеся файлы

Задание 3. Сохранение изображений (графиков).

Реализована функция сохранения графиков Matplotlib в PNG с высоким разрешением. В качестве примера построена и сохранена диаграмма рассеяния зависимости y от x2.

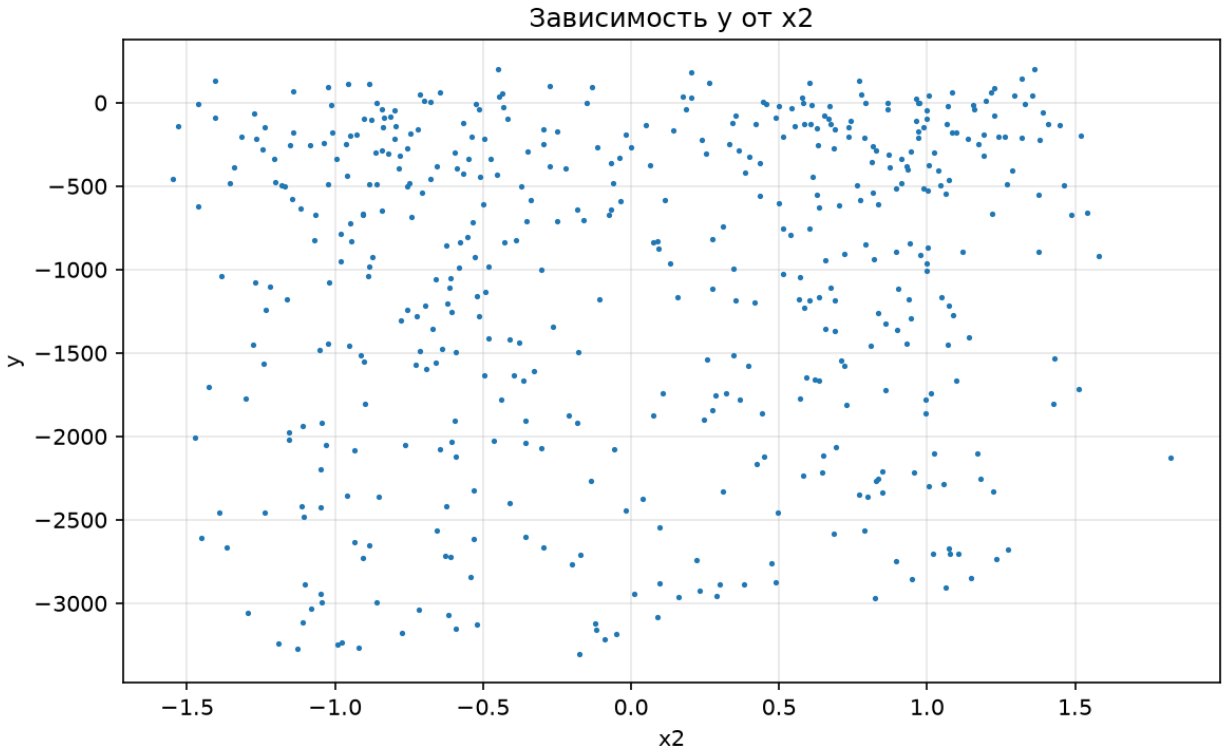


Рисунок 3 – Пример сохранения графика

Задание 4. Проверка данных.

Реализована функция проверки качества данных: она подсчитывает пропуски по каждому столбцу, выводит типы данных и находит некорректные числовые значения. В датасете обнаружены пропуски в столбцах x1 и x2, а также строка «ошибка» в столбце x1, из-за которой столбец имеет тип object.

```
============================================================
ЗАДАНИЕ 4. ПРОВЕРКА ДАННЫХ
============================================================

Пропущенные значения:
{'id': 0, 'timestamp': 0, 'category': 0, 'x1': 0, 'x2': 10, 'x4': 0, 'y': 0}

Типы данных:
{'id': 'int64', 'timestamp': 'str', 'category': 'str', 'x1': 'str', 'x2': 'float64', 'x4': 'float64', 'y': 'float64'}

Некорректные числовые значения:
{}
```

Рисунок 4 – Проверка на пропущенные значения

Задание 5. Вывод таблицы.

Реализована функция вывода первых n строк таблицы, её размера и типов столбцов.

```
============================================================
ЗАДАНИЕ 5. ВЫВОД ТАБЛИЦЫ
============================================================
   id            timestamp category                  x1        x2        x4           y
0   1  2024-01-01 00:00:00        A   0.6835119515543189  0.175178 -0.809604   38.302431
1   2  2024-01-01 01:00:00        A   3.7523416784317725  0.202209  0.425061  181.422702
2   3  2024-01-01 02:00:00        B   1.9008476763721005  0.769198 -0.469839  132.614133
3   4  2024-01-01 03:00:00        A  -1.1538073113248062  1.154333 -0.005452  -16.476770
4   5  2024-01-01 04:00:00        C   -1.796829342696716  1.227887  0.562284  -76.589107

Shape: (500, 7)
Dtypes:
id             int64
timestamp        str
category         str
x1               str
x2           float64
x4           float64
y            float64
dtype: object
```

Рисунок 5 – Вывод таблицы

Задание 6. Извлечение столбцов по варианту.

Реализована функция извлечения подмножества столбцов. В датасете нет столбца x3, поэтому для анализа выбраны реально существующие: x1, x2, x4, y.

```
============================================================
ЗАДАНИЕ 6. ИЗВЛЕЧЕНИЕ СТОЛБЦОВ
============================================================
                   x1        x2           y
0   0.6835119515543189  0.175178   38.302431
1   3.7523416784317725  0.202209  181.422702
2   1.9008476763721005  0.769198  132.614133
3  -1.1538073113248062  1.154333  -16.476770
4   -1.796829342696716  1.227887  -76.589107
```

Рисунок 6 – Вывод извлеченных столбцов

Задание 7. Явные типы данных.

Реализована функция приведения столбцов к заданным типам. Числовые признаки приведены к float64.

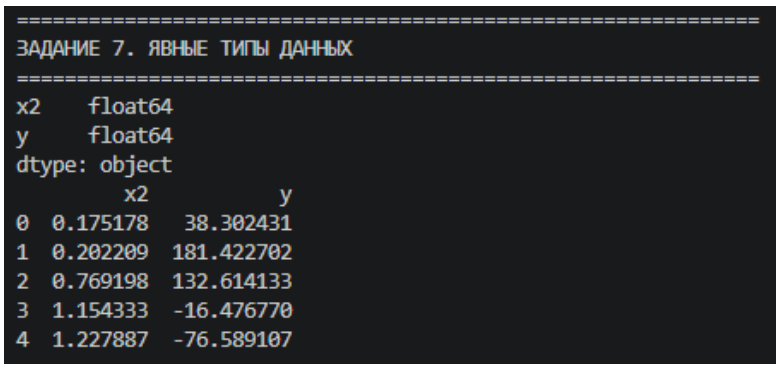


Рисунок 7 – Вывод приведенных столбцов

Раздел 2

Задание 8. Обработка проблемных данных.

Реализована функция, поддерживающая стратегии drop, cast, mean, ffill, bfill, zero, value. В столбце x1 строка «ошибка» сначала превращена в NaN (стратегия cast), затем пропуски заполнены средним значением (стратегия mean).

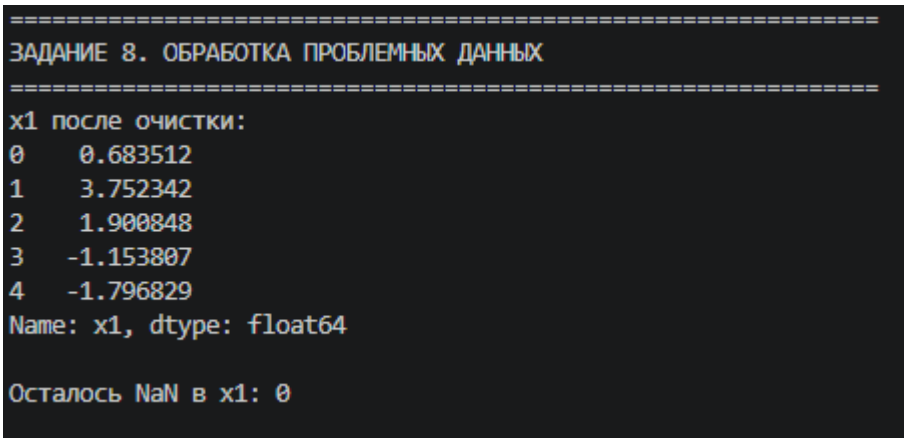


Рисунок 8 – Заполнение пропусков

Задание 9. DataFrame → NumPy.

Реализована функция преобразования DataFrame в массив NumPy. Полученный массив имеет форму (n, 4) и тип float64.

```
============================================================
ЗАДАНИЕ 9. DATAFRAME -> NUMPY
============================================================
Тип: <class 'numpy.ndarray'>
Shape: (490, 4)
[[ 6.83511952e-01  1.75178456e-01 -8.09603520e-01  3.83024310e+01]
 [ 3.75234168e+00  2.02208891e-01  4.25061044e-01  1.81422702e+02]
 [ 1.90084768e+00  7.69198036e-01 -4.69839347e-01  1.32614133e+02]
 [-1.15380731e+00  1.15433284e+00 -5.45227088e-03 -1.64767699e+01]
 [-1.79682934e+00  1.22788677e+00  5.62284248e-01 -7.65891072e+01]]
```

Рисунок 9 – Преобразование в массив

Задание 10. Функции масштабирования.

Реализованы собственные классы MinMaxScalerCustom и StandardScalerCustom с методами fit, transform, fit_transform, inverse_transform. Проведено масштабирование исходного массива обоими способами.

```
============================================================
ЗАДАНИЕ 10. МАСШТАБИРОВАНИЕ
============================================================
MinMax (первые 5 строк):
[[0.52610197 0.51134446 0.00144849 0.95381688]
 [0.79176621 0.51936559 0.00639876 0.99467738]
 [0.631485   0.68761641 0.00281074 0.98074265]
 [0.36704784 0.80190298 0.00467266 0.93817755]
 [0.31138234 0.82372969 0.00694895 0.92101562]]

StandardScaler (первые 5 строк):
[[ 0.33547939  0.17766022 -1.12022748  1.14909669]
 [ 1.86829323  0.20978368 -1.10359078  1.29536088]
 [ 0.94351216  0.88360382 -1.11564927  1.24548014]
 [-0.5822217   1.34130498 -1.10939181  1.09311416]
 [-0.90339725  1.4287178  -1.10174174  1.03168133]]
```

Рисунок 10 – Масштабирование

Задание 11. Восстановление масштаба.

Проверено, что inverse_transform возвращает исходные значения с точностью до 1e-10.

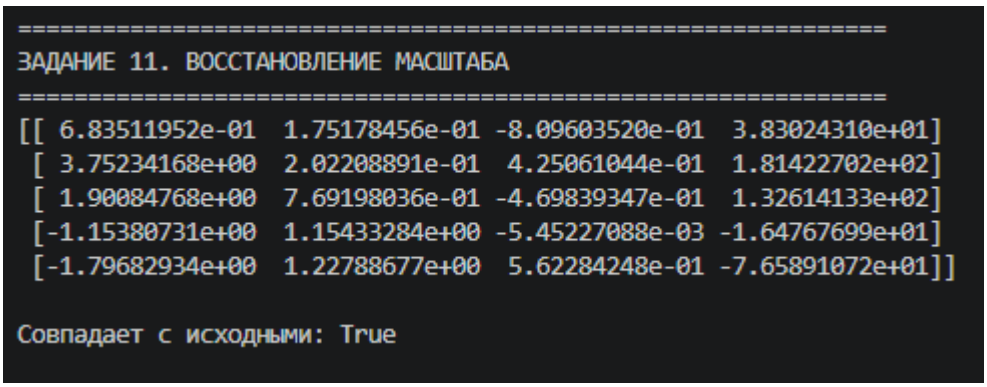


Рисунок 11 – Восстановление масштаба

Задание 12. Разбиение на 3 части.

Массив разбит на train / validation / test в пропорции 70 / 15 / 15 без перемешивания, чтобы сохранить порядок.

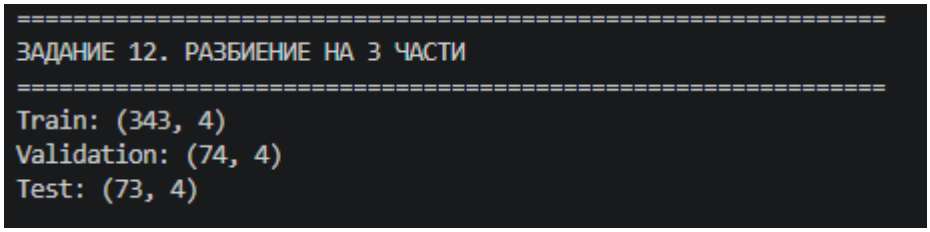


Рисунок 12 – Разбиение данных на части

Задание 13. Фиксация seed.

Реализована функция, фиксирующая состояние всех генераторов случайных чисел (random, numpy, TensorFlow, PyTorch). Проверено, что два последовательных вызова дают одинаковые массивы случайных чисел.

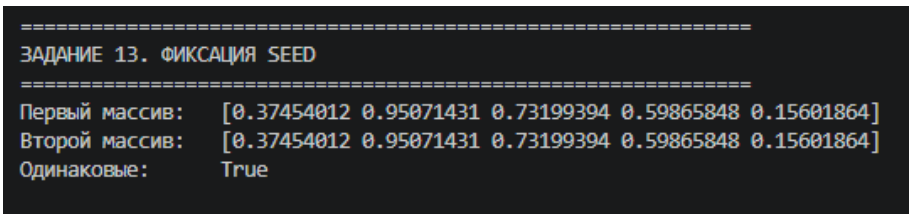


Рисунок 13 – Фиксация seed

Раздел 3

Задание 14. График исходных данных.

Построен линейный график всех четырёх столбцов массива X.

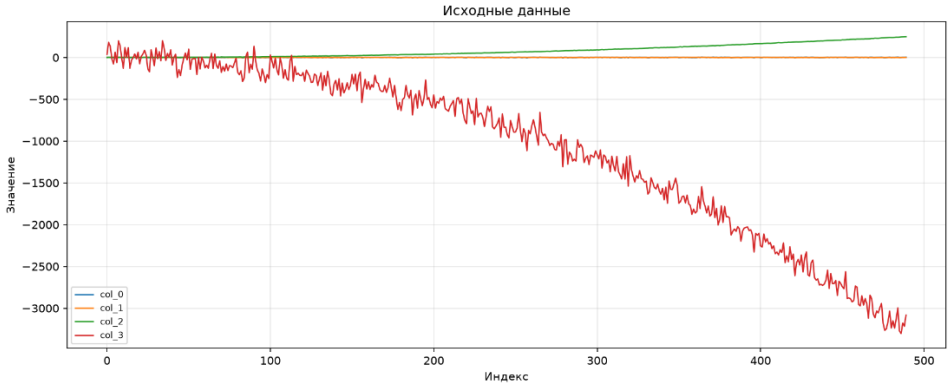


Рисунок 14 – Линейный график всех столбцов массива

Задание 15. Гистограмма и плотность.

Построена нормализованная гистограмма столбца x2.

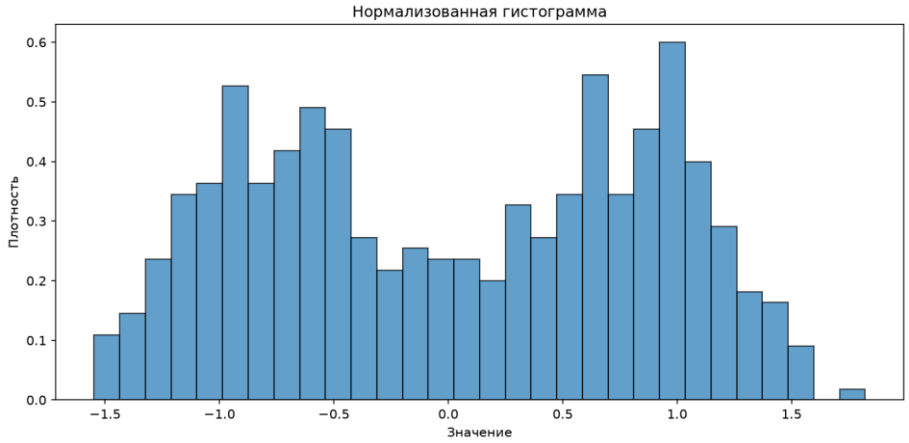


Рисунок 15 - Гистограмма

Задание 16. Отсортированные столбцы.

Выведены отсортированные значения x2 (первые 15).

```
============================================================
ЗАДАНИЕ 16. СОРТИРОВКА
============================================================
Отсортированные значения x2 (первые 15):
[-1.54800342 -1.53121426 -1.4747505  -1.46243712 -1.46130653 -1.45001478
 -1.42559295 -1.40382521 -1.40359561 -1.3926516  -1.38370154 -1.3642829
 -1.35534391 -1.34061818 -1.31603314]
```

Рисунок 16 – Сортировка данных

Задание 17. Эмпирическая функция распределения.

Построена ECDF для x2.

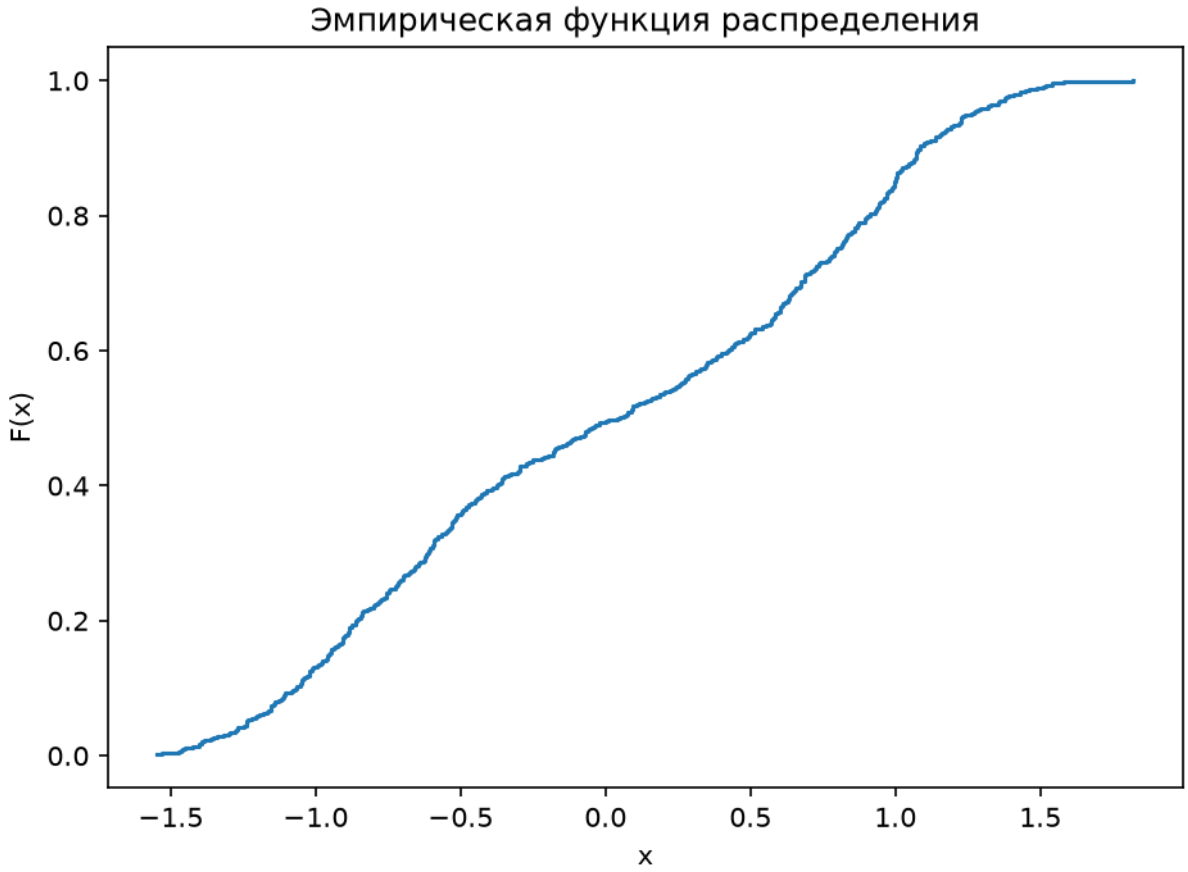


Рисунок 17 – График распределения

Задание 18. Статистики.

Вычислены среднее, дисперсия, мода и медиана для x2.

```
============================================================
ЗАДАНИЕ 18. СТАТИСТИКИ
============================================================
  mean: 0.02568542082404832
  var: 0.7080460602948297
  mode: -1.548003418133002
  median: 0.0448501088557536
```

Рисунок 18 – Вычисление метрик

Задание 19. Доверительные интервалы.

Построены доверительные интервалы для среднего (через распределение Стьюдента) и для дисперсии (через распределение хи-квадрат) при уровне значимости 0,05.

```
============================================================
ЗАДАНИЕ 19. ДОВЕРИТЕЛЬНЫЕ ИНТЕРВАЛЫ
============================================================
Интервал для среднего:  (np.float64(-0.04907997914399774), np.float64(0.10045082079209437))
Интервал для дисперсии: (np.float64(0.6283305253627959), np.float64(0.8075534793793682))
```

Рисунок 19 – Доверительные интервалы

Задание 20. Ковариация и корреляция.

Вычислены ковариационная и корреляционная матрицы массива X.

```
============================================================
ЗАДАНИЕ 20. КОВАРИАЦИЯ И КОРРЕЛЯЦИЯ
============================================================
Ковариационная матрица:
[[ 4.01655741e+00 -4.82086756e-02  1.21616039e+01  1.97390521e+01]
 [-4.82086756e-02  7.09494007e-01 -5.85703202e+00  9.32880527e+01]
 [ 1.21616039e+01 -5.85703202e+00  5.51887892e+03 -7.24348619e+04]
 [ 1.97390521e+01  9.32880527e+01 -7.24348619e+04  9.59430435e+05]]

Корреляционная матрица:
[[ 1.         -0.02855774  0.08168427  0.01005524]
 [-0.02855774  1.         -0.09360042  0.11306936]
 [ 0.08168427 -0.09360042  1.         -0.99544085]
 [ 0.01005524  0.11306936 -0.99544085  1.        ]]
```

Рисунок 20 - Ковариационная и корреляционная матрицы

Задание 21. Значимость коэффициента корреляции.

Проверена значимость корреляции между x2 и y с помощью критерия Пирсона. Получены коэффициент корреляции и p-value.

```
============================================================
ЗАДАНИЕ 21. ЗНАЧИМОСТЬ КОРРЕЛЯЦИИ
============================================================
r = 0.113069
p-value = 0.012261
Корреляция значима.
```

Рисунок 21 – Значимость корреляции

Задание 22. Взаимная корреляция.

Рассчитана взаимная корреляция между x2 и y для лагов от –50 до 50.

```
============================================================
ЗАДАНИЕ 22. ВЗАИМНАЯ КОРРЕЛЯЦИЯ
============================================================
Лаги (первые 10):      [-50 -49 -48 -47 -46 -45 -44 -43 -42 -41]
Корреляции (первые 10): [0.0932 0.097  0.0954 0.0919 0.0866 0.0811 0.0738 0.0617 0.0544 0.0453]
```

Рисунок 22 – Взаимная корреляция

Задание 23. Производная и градиент.

Вычислена производная x2 и многомерный градиент массива X.

```
============================================================
ЗАДАНИЕ 23. ПРОИЗВОДНАЯ И ГРАДИЕНТ
============================================================
Производная x2 (первые 10): [ 0.02703044  0.29700979  0.47606197  0.22934437  0.03079703  0.08078569
  0.07252691 -0.0348687  -0.19410648 -0.35748236]
Градиент X (первые 3 строки):
[[ 3.06882973e+00  2.70304358e-02  1.23466456e+00  1.43120271e+02]
 [ 6.08667862e-01  2.97009790e-01  1.69882086e-01  4.71558512e+01]
 [-2.45307449e+00  4.76061972e-01 -2.15256658e-01 -9.89497360e+01]]
```

Рисунок 23 – Производная и градиент

Задание 24. Свёртка.

Выполнена свёртка x2 и y в режиме full.

```
============================================================
ЗАДАНИЕ 24. СВЁРТКА
============================================================
Свёртка (первые 20 значений): [  6.7097607   39.52644087  89.37857721 207.69312173 341.71125042
 405.39856018 360.84185289 428.30872938 459.19810627 530.16653588
 564.51160793 551.03086276 445.44295566 573.06141048 559.08826189
 596.3226262  521.36522175 476.00393511 432.33315539 675.5359009 ]
```

Рисунок 24 - Свёртка

Задание 25. Скалярное и векторное произведения.

Вычислены скалярное и векторное произведения для первых трёх компонент x2 и y.

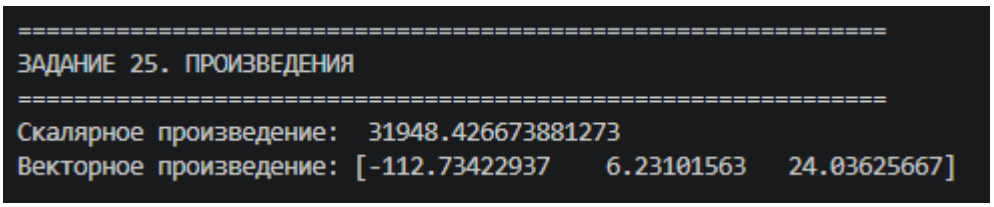


Рисунок 25 – Произведения

Задание 26. Нормы L1 и L2.

Вычислены L1- и L2-нормы для x2.

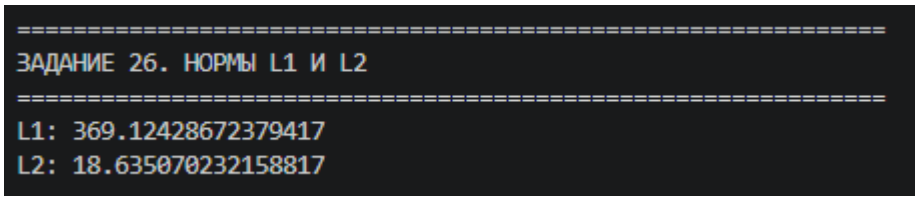


Рисунок 26 - Нормы

Задание 27. Проверка гипотез о распределении.

Проверены гипотезы: для x1 — о равномерном распределении (критерий Колмогорова–Смирнова), для x2 — о нормальном распределении (Колмогоров–Смирнов и Шапиро–Уилк).

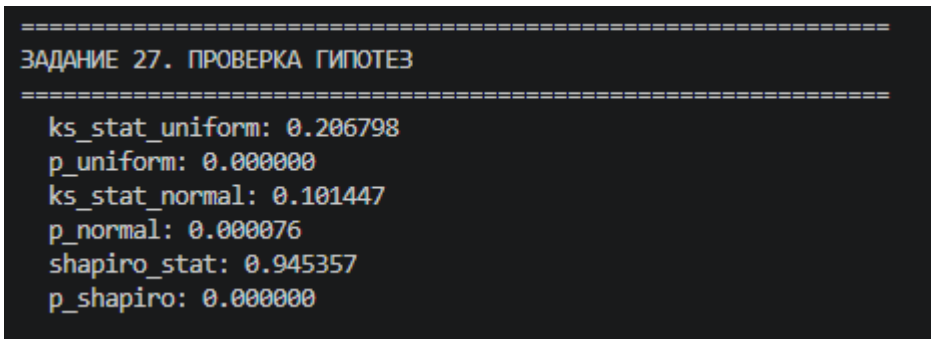


Рисунок 27 – Гипотезы о распределении

Задание 28. Спектрограмма.

Построена спектрограмма для x2.

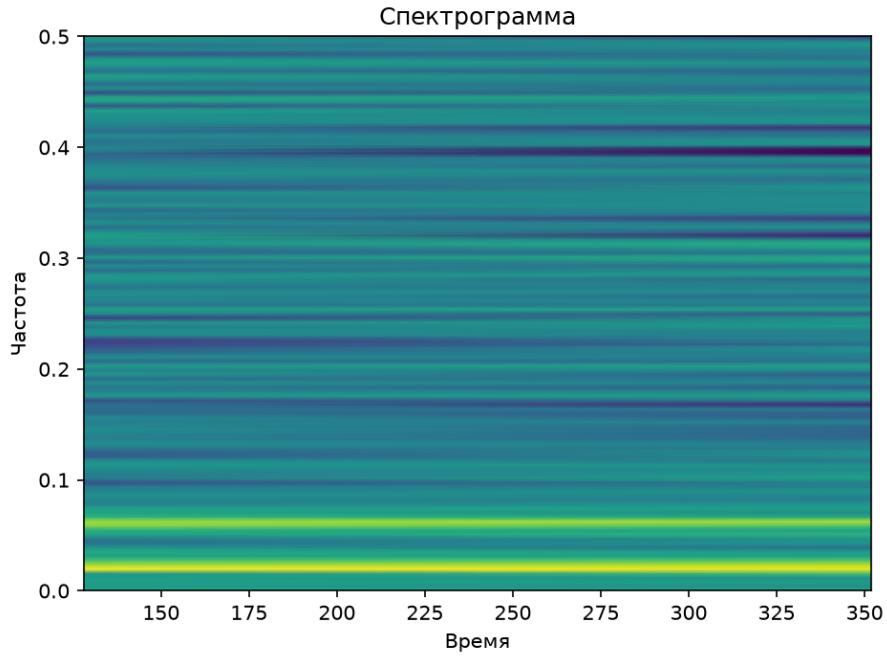


Рисунок 28 - Спектрограмма

Задание 29. Периодограмма.

Построена периодограмма для x2.

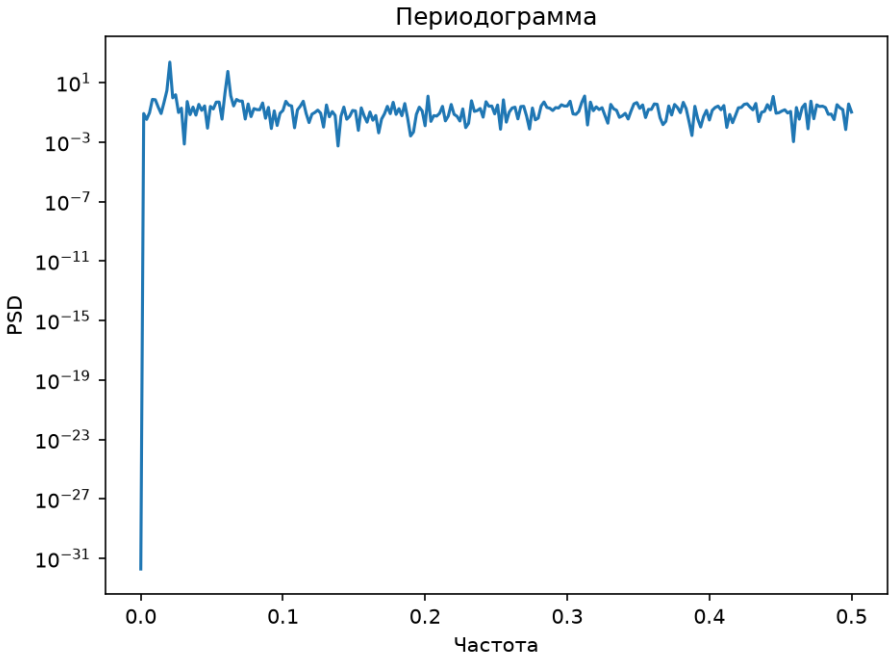


Рисунок 29 - Периодограмма

Задание 30. АЧХ через FFT.

Построен амплитудный спектр через быстрое преобразование Фурье.

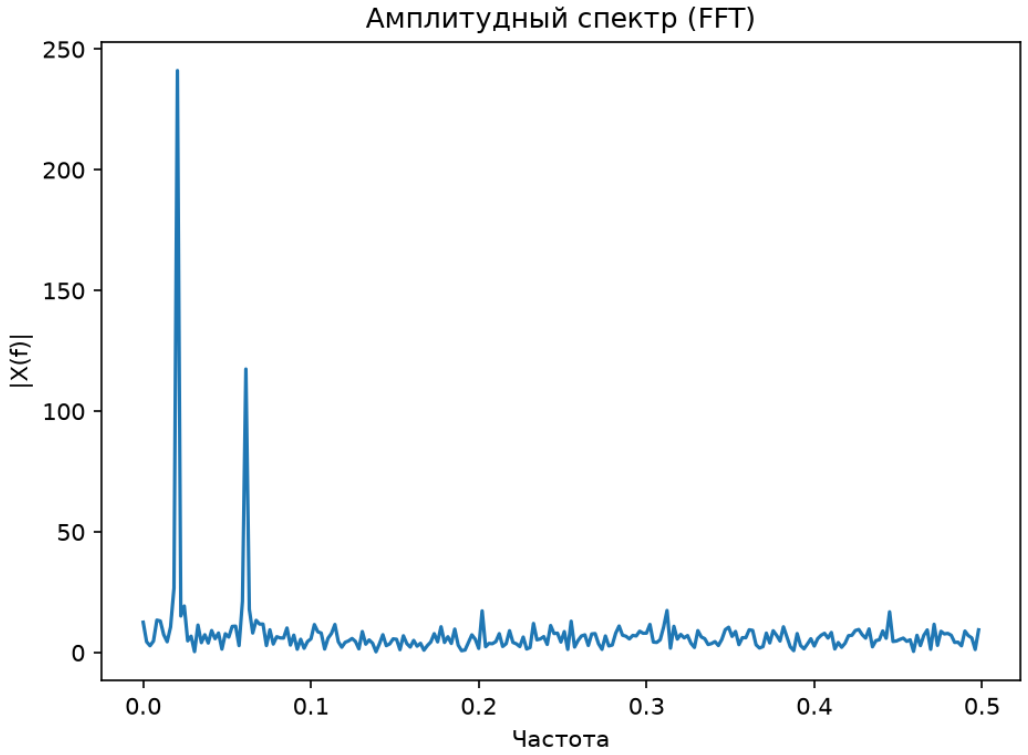


Рисунок 30 - АЧХ

Задание 31. Кубическая интерполяция.

Выполнена кубическая интерполяция x2 с увеличением числа точек в 10 раз.

```
============================================================
ЗАДАНИЕ 31. КУБИЧЕСКАЯ ИНТЕРПОЛЯЦИЯ
============================================================
Исходных точек: 490
Новых точек:    4900
```

Рисунок 31 - Кубическая интерполяция

Задание 32. Интерполяция сплайнами.

Выполнена сплайн-интерполяция x2 (k = 3, s = 0).

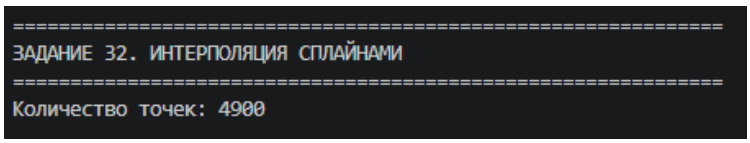


Рисунок 32 - Интерполяция сплайнами

Задание 33. Бинарные маски.

Построены маски: положительные, отрицательные, нулевые, попадающие в интервал [–1; 1].

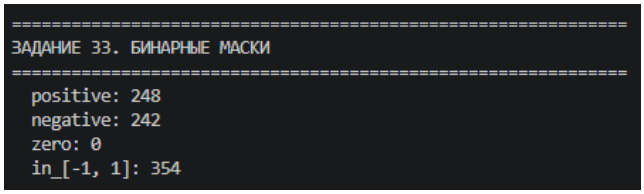


Рисунок 33 – Бинарные маски

Задание 34. Сравнение собственного и sklearn-масштабирования.

Проведено сравнение собственных классов MinMaxScalerCustom и StandardScalerCustom с аналогами из sklearn. Результаты совпадают с точностью до 1e-10. Таблица сравнения сохранена в файл.

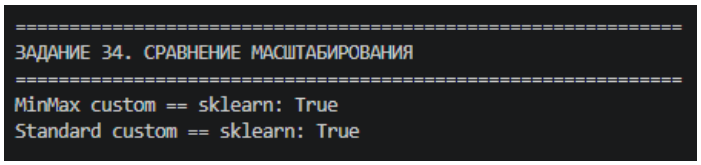


Рисунок 34 – Сравнение классов

Задание 35. Инверсия.

Проверено, что обратное преобразование восстанавливает исходные данные и для собственного класса, и для sklearn.

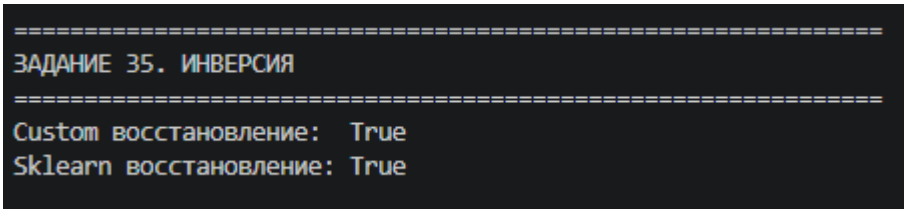


Рисунок 35 - Инверсия

Задание 36. Скользящее окно.

Построена матрица окон ширины 5 для x2.

```
============================================================
ЗАДАНИЕ 36. СКОЛЬЗЯЩЕЕ ОКНО
============================================================
Shape: (486, 5)
[[0.17517846 0.20220889 0.76919804 1.15433284 1.22788677]
 [0.20220889 0.76919804 1.15433284 1.22788677 1.21592689]
 [0.76919804 1.15433284 1.22788677 1.21592689 1.38945815]
 [1.15433284 1.22788677 1.21592689 1.38945815 1.36098071]
 [1.22788677 1.21592689 1.38945815 1.36098071 1.31972076]]
```

Рисунок 36 – Скользящее окно

Задание 37. Скользящее среднее.

Вычислено скользящее среднее ширины 5.

```
======================================================================
ЗАДАНИЕ 37. СКОЛЬЗЯЩЕЕ СРЕДНЕЕ
======================================================================
Shape: (486,)
[0.705761   0.91391068 1.15136054 1.26971707 1.30279466 1.25177085
 1.12953668 0.97213322 0.87357543 0.66213835]
```

Рисунок 37 - Скользящее среднее

Задание 38. Тензор TensorFlow.

Создан тензор X_tf из массива X.

Вставить: скриншот вывода тензора и его формы.

```
============================================================
ЗАДАНИЕ 38. ТЕНЗОР TENSORFLOW
============================================================
I0000 00:00:1791269478.479007    8500 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
tf.Tensor(
[[ 6.8351197e-01  1.7517845e-01 -8.0960351e-01  3.8302429e+01]
 [ 3.7523417e+00  2.0220889e-01  4.2506105e-01  1.8142270e+02]
 [ 1.9008477e+00  7.6919806e-01 -4.6983933e-01  1.3261414e+02]
 ...
 [ 2.0883217e+00 -7.7371609e-01  2.4696034e+02 -3.1775527e+03]
 [ 1.3637830e+00 -8.7367907e-02  2.4803680e+02 -3.2159512e+03]
 [ 3.6934147e+00  8.9787774e-02  2.4824251e+02 -3.0819236e+03]], shape=(490, 4), dtype=float32)
Shape: (490, 4)
```

Рисунок 38 – Тензор TensorFlow

Задание 39. Тензор PyTorch.

Создан тензор X_pt из массива X.

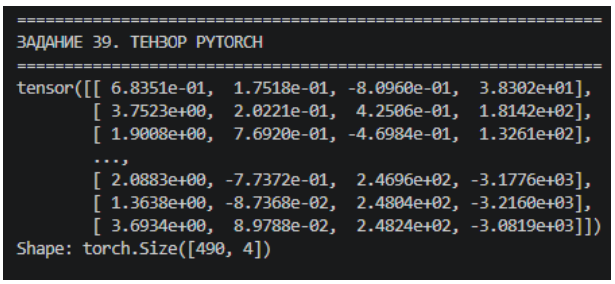


Рисунок 39 - Тензор PyTorch

Задание 40. Случайные тензоры.

Созданы случайные тензоры A (k × n), W (m × p) и B (k × p).

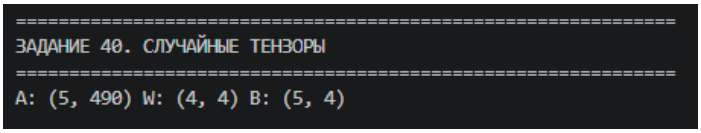


Рисунок 40 - Случайные тензоры

Задание 41. Операция AXW+B (TensorFlow).

Вычислено произведение A @ X @ W + B средствами TensorFlow.

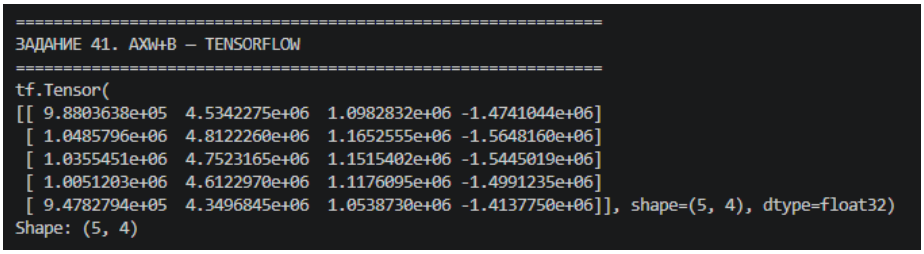


Рисунок 41 - AXW+B TensorFlow

Задание 42. Операция AXW+B (PyTorch).

Та же операция выполнена в PyTorch.

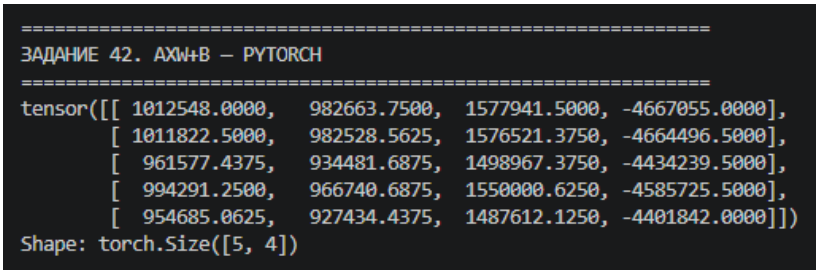


Рисунок 42 - AXW+B PyTorch

Задание 43. Матрицы T, P, Q.

Созданы три случайные матрицы формы 3 × 10.

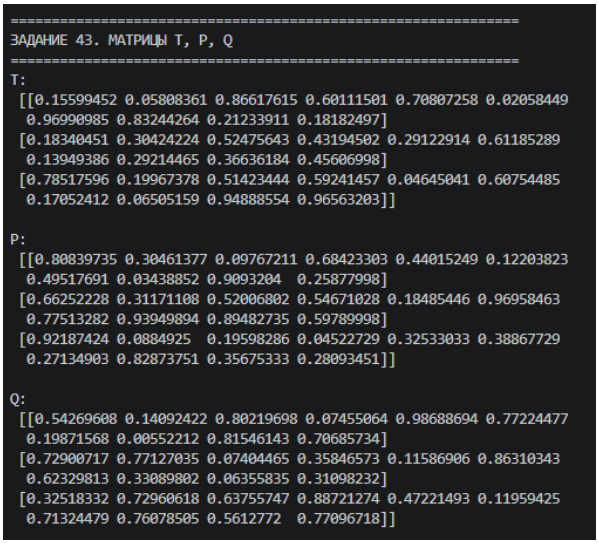


Рисунок 43 – Случайные матрицы

Задание 44. Операция в Keras.

Вычислено выражение V = |sin(T) − exp(P) · √Q| средствами Keras.

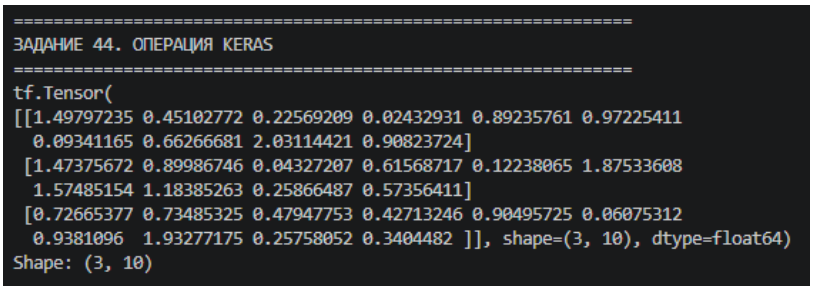


Рисунок 44 – Вычисление выражения в Keras

Задание 45. Та же операция в PyTorch.

Результат повторён в PyTorch.

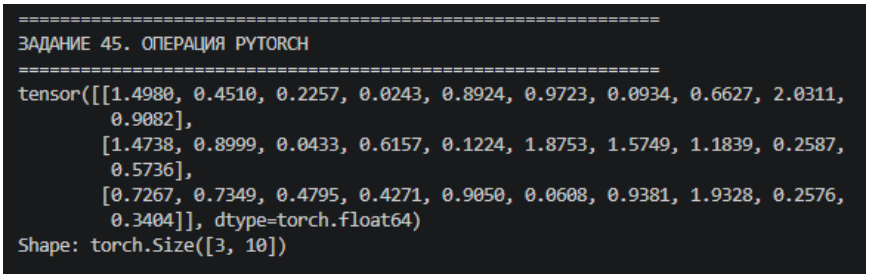


Рисунок 45 – Вычисление того же выражения в PyTorch

Задание 46. Разреженный массив SciPy.

Создана разреженная матрица размером 100 000 × 100 000 с плотностью 1e-5. Выведены её размер, число ненулевых элементов, тип, формат и плотность.

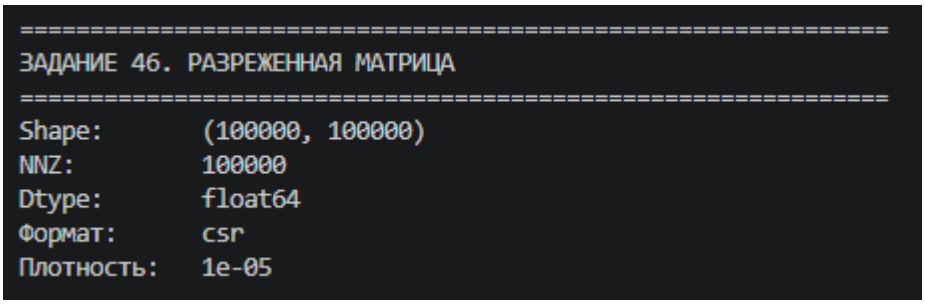


Рисунок 46 – Разреженная матрица

Задание 47. Обратная матрица.

Создана случайная матрица 100 × 100, найдены её определитель и обратная матрица. Проверено условие $M @ M^{-1} = I$.

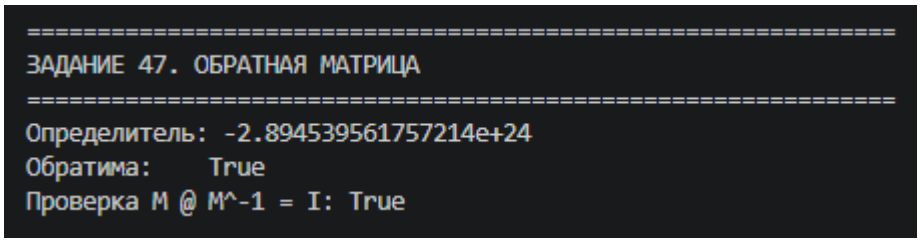


Рисунок 47 – Обратная матрица

Задание 48. Достроение массива F.

Собран трёхмерный массив F формы (n, m, 3): [исходные данные, Min-Max, StandardScaler]. Для PCA выполнено преобразование к форме (n, m·3).

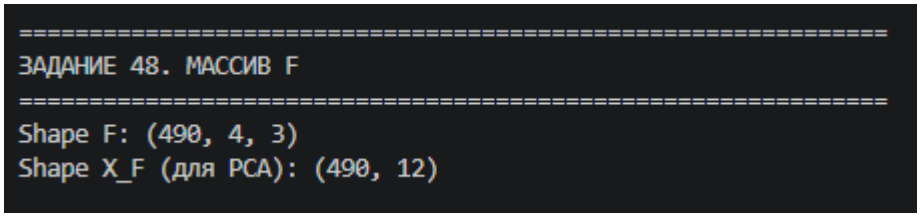


Рисунок 48 – Достроение массива

Задание 49. Пайплайн sklearn + PCA.

Для двух пайплайнов (MinMax + PCA и Standard + PCA) вычислены значения логарифмического правдоподобия. Построен график кумулятивной объяснённой дисперсии в логарифмическом масштабе.

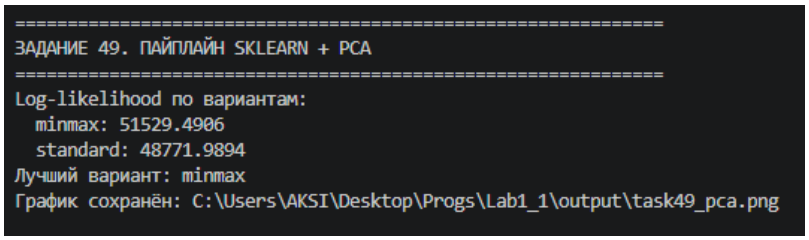


Рисунок 49 – Вычисление значений логарифмического правдоподобия

Задание 50. Обработка изображения средствами SciPy.

Для цветного изображения image.jpg выполнены десять операций: разделение на каналы R, G, B; поворот на 40° с бикубической интерполяцией; перевод в оттенки серого; построение гистограммы яркостей чёрно-белого изображения (256 бинов); разбиение изображения на три части по квантилям гистограммы; наложение чёрной круговой рамки; добавление гауссовского шума ($\sigma = 25$); гауссовское размытие ($\sigma = 2$); операция sharpening через unsharp masking (amount = 1,5); PCA к чёрно-белому изображению (32 компоненты), построение кривой MSE и сетки из 16 первых главных компонент. Все обработанные изображения сохранены отдельными PNG-файлами, а также в виде сравнительных фигур «до и после».



Рисунок 50 – Оригинал изображения



Рисунок 51 – Изображения в разных каналах

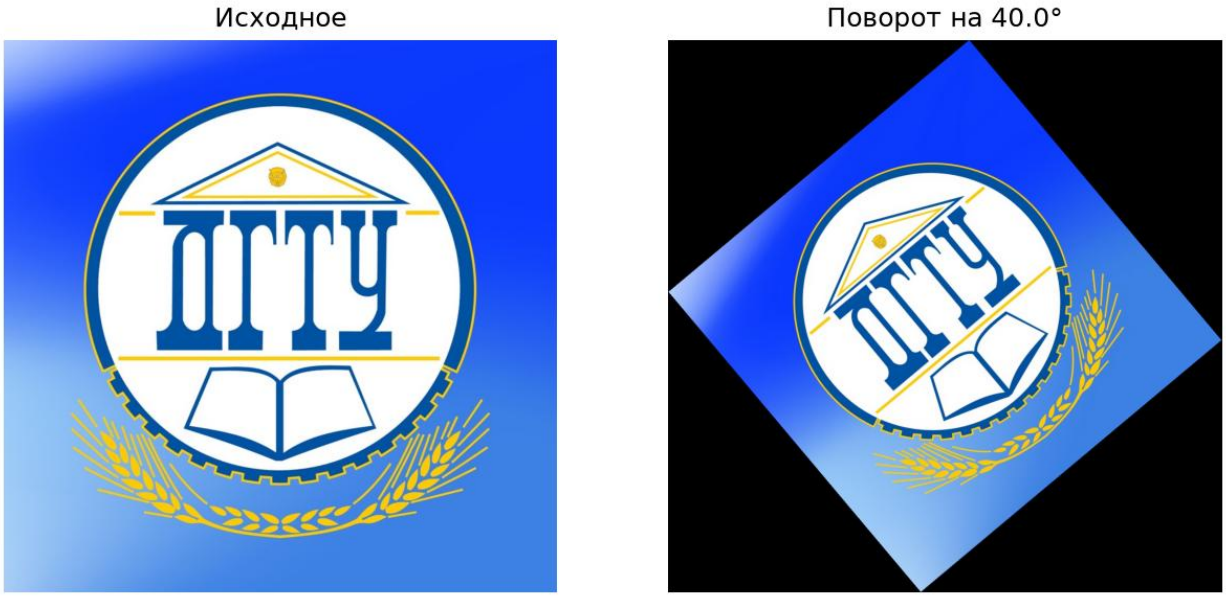


Рисунок 52 – Повернутое изображение

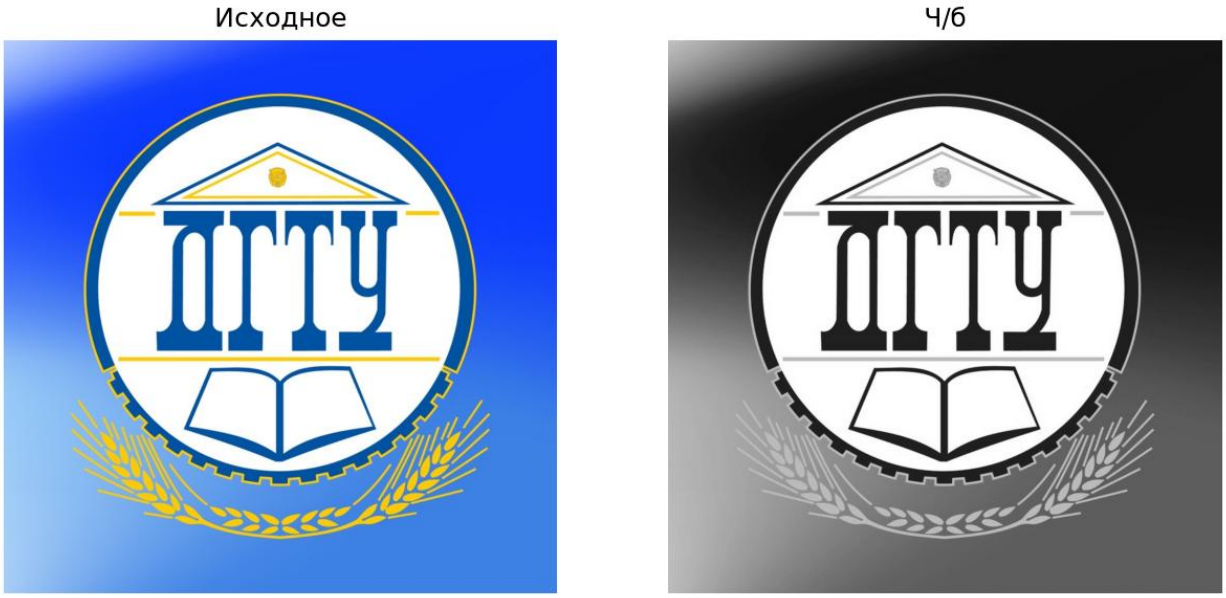


Рисунок 53 – Черно-белое изображение

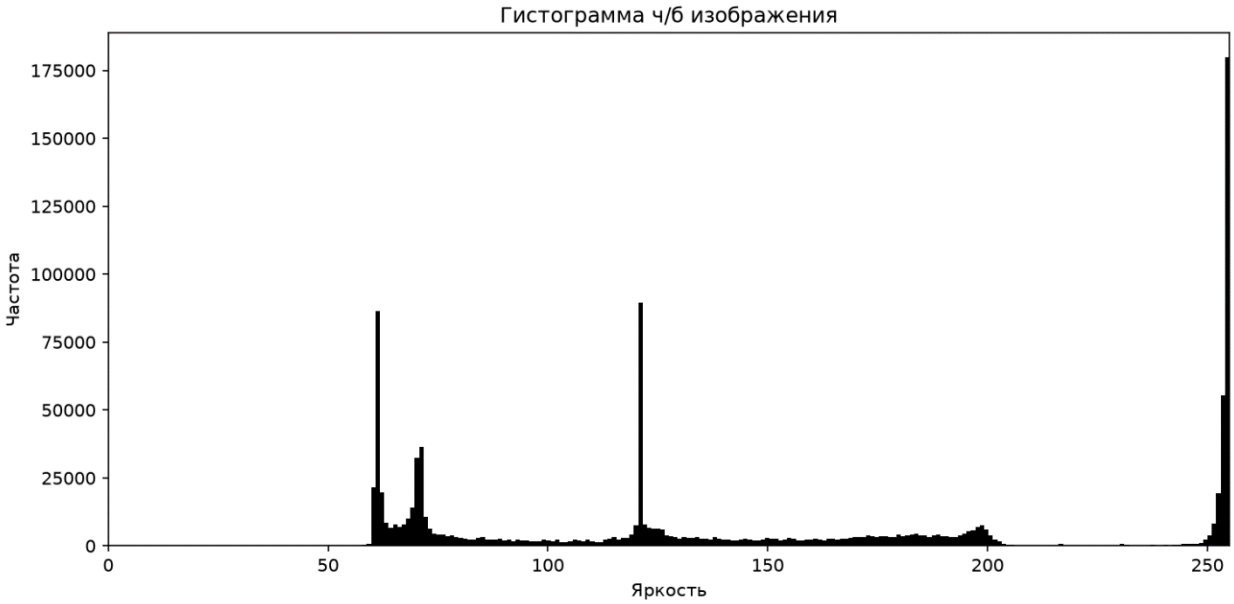


Рисунок 54 – Гистограмма ч/б изображения

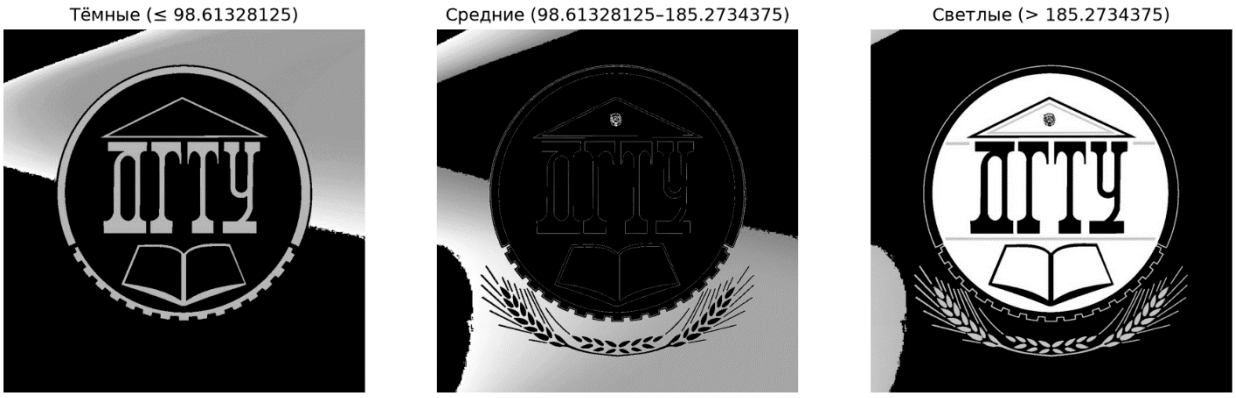


Рисунок 55 – Яркость участков

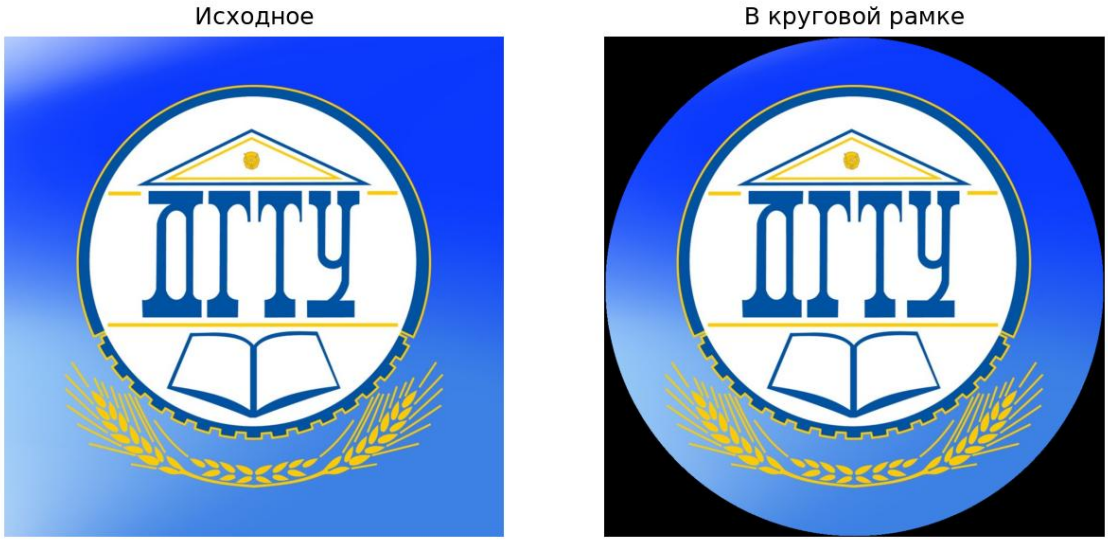


Рисунок 56 – Добавление рамки на изображение



Рисунок 57 – Добавление шума на изображение

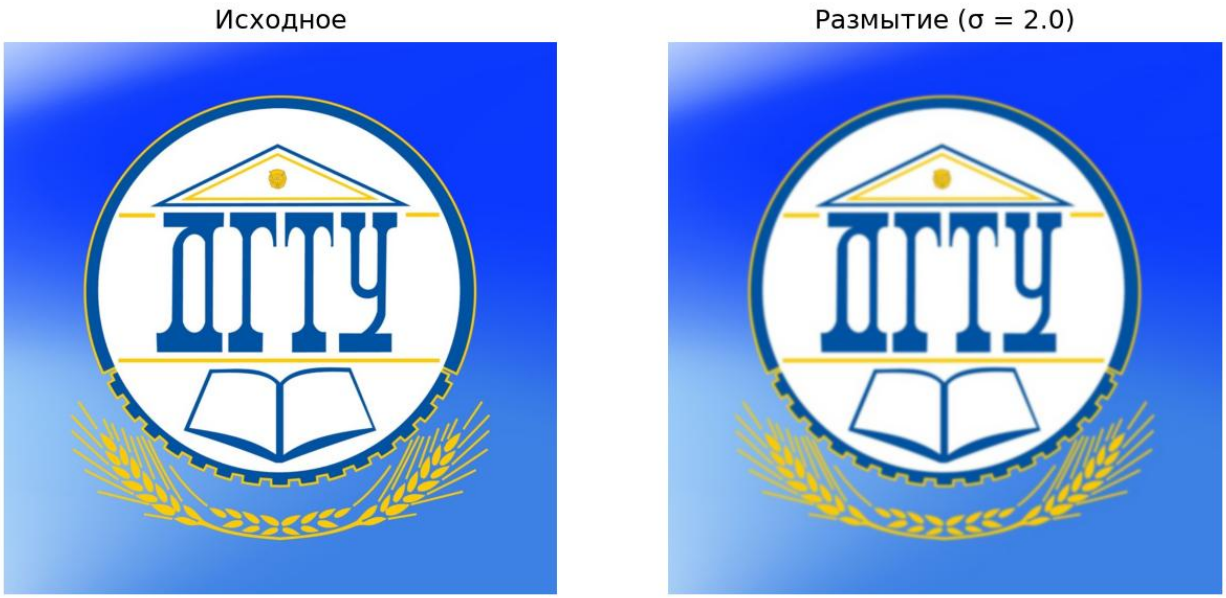


Рисунок 58 – Добавление размытия на изображение

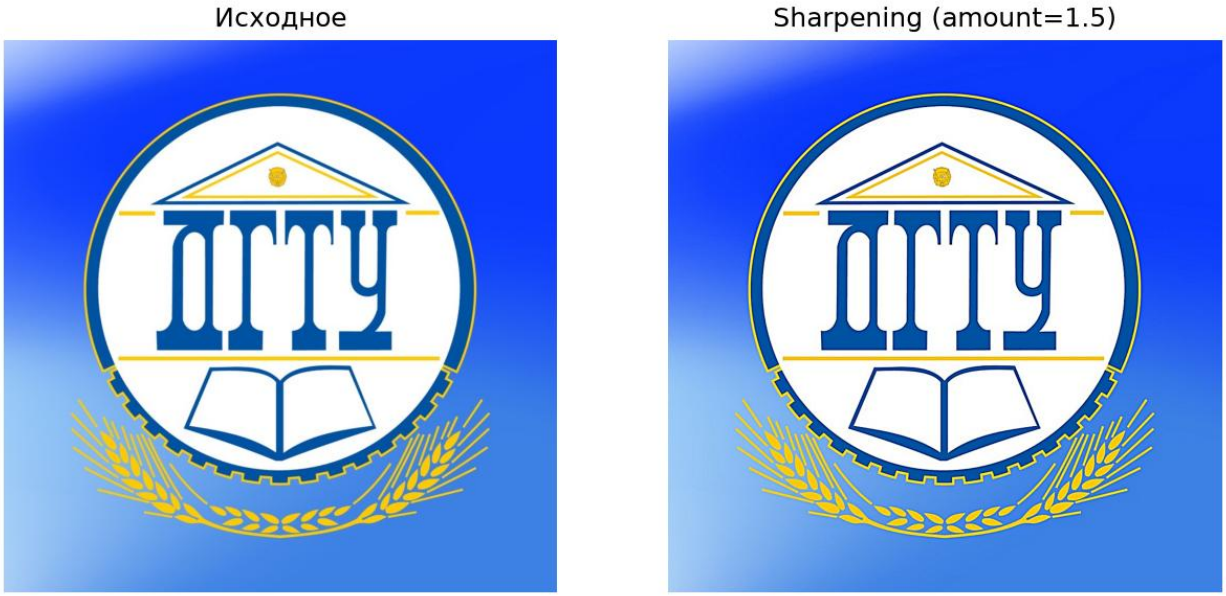


Рисунок 59 – Повышение резкости на изображении

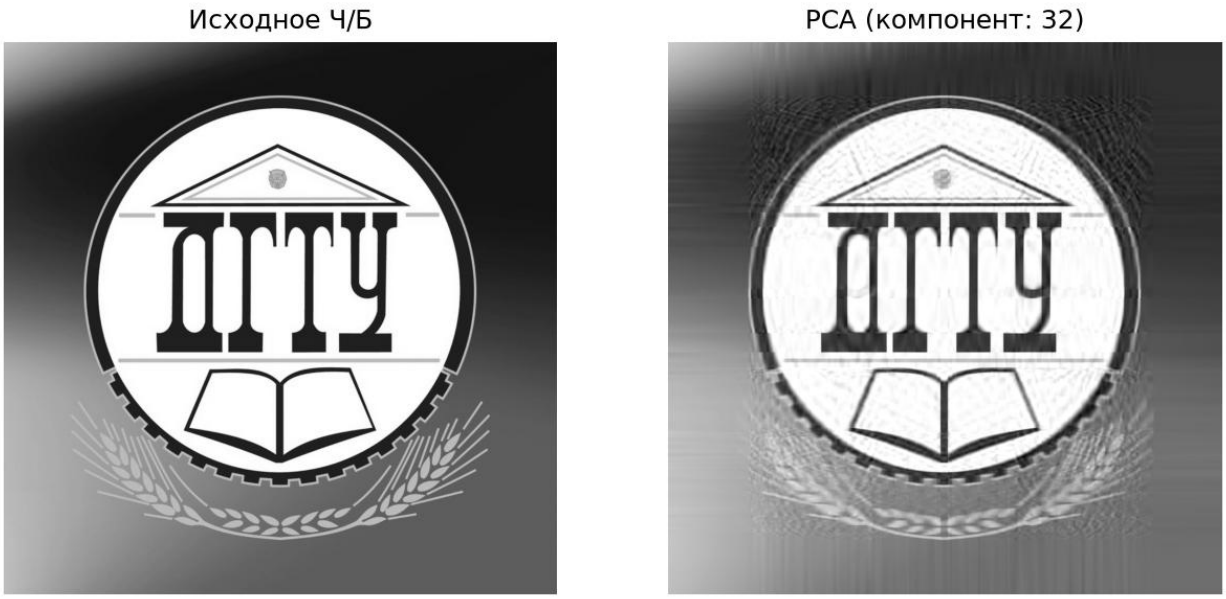


Рисунок 60 – Сжатое ч/б изображение

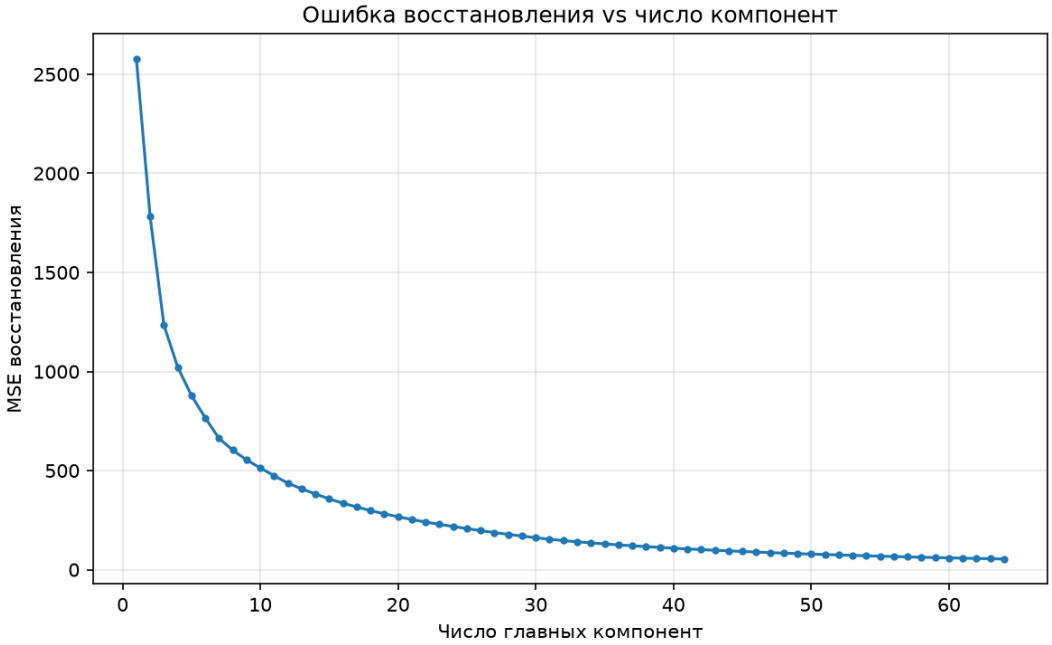
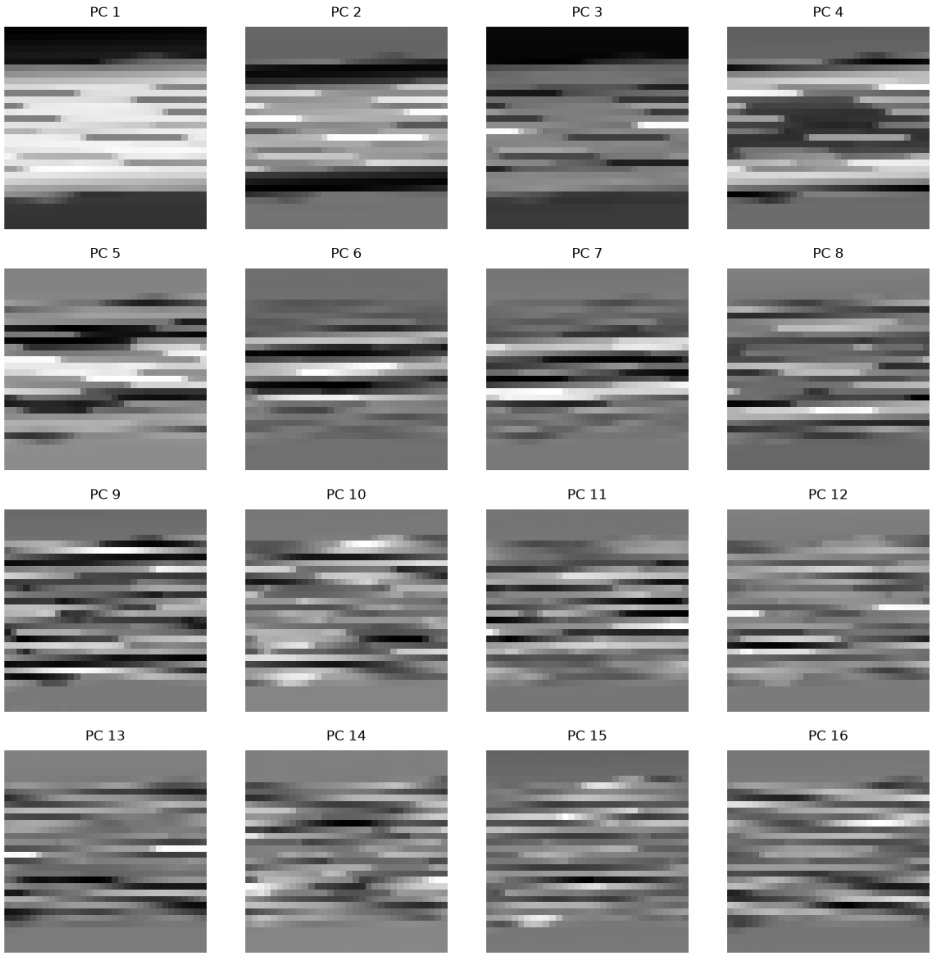


Рисунок 61 – Ошибка восстановления относительно числа компонент



Отправление выполненного задания на GitHub.

Был создан удаленный репозиторий, а после запушены все файлы выполненной лабораторной.

```
AKSI@HOME-PC MINGW64 ~
$ cd Desktop/

AKSI@HOME-PC MINGW64 ~/Desktop
$ cd Progs/

AKSI@HOME-PC MINGW64 ~/Desktop/Progs
$ cd Lab1_1

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1
$ git init
Initialized empty Git repository in C:/Users/AKSI/Desktop/Progs/Lab1_1/.git/

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ nano README.md

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git add README.md

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git commit -m "Первый коммит, добавлен README"
[main (root-commit) 39bb221] Первый коммит, добавлен README
 1 file changed, 97 insertions(+)
 create mode 100644 README.md

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git config user.name
7AKSI7

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git config user.email
solodkijmaksim64@gmail.com

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git remote add origin https://github.com/7AKSI7/MachineLearningLab1.git

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 2.71 KiB | 2.71 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/7AKSI7/MachineLearningLab1.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

AKSI@HOME-PC MINGW64 ~/Desktop/Progs/Lab1_1 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data/
        main.py
        output/
        requirements.txt
        src/

nothing added to commit but untracked files present (use "git add" to track)
```

Рисунок 62 – Первый коммит и отправка на удаленный репозиторий

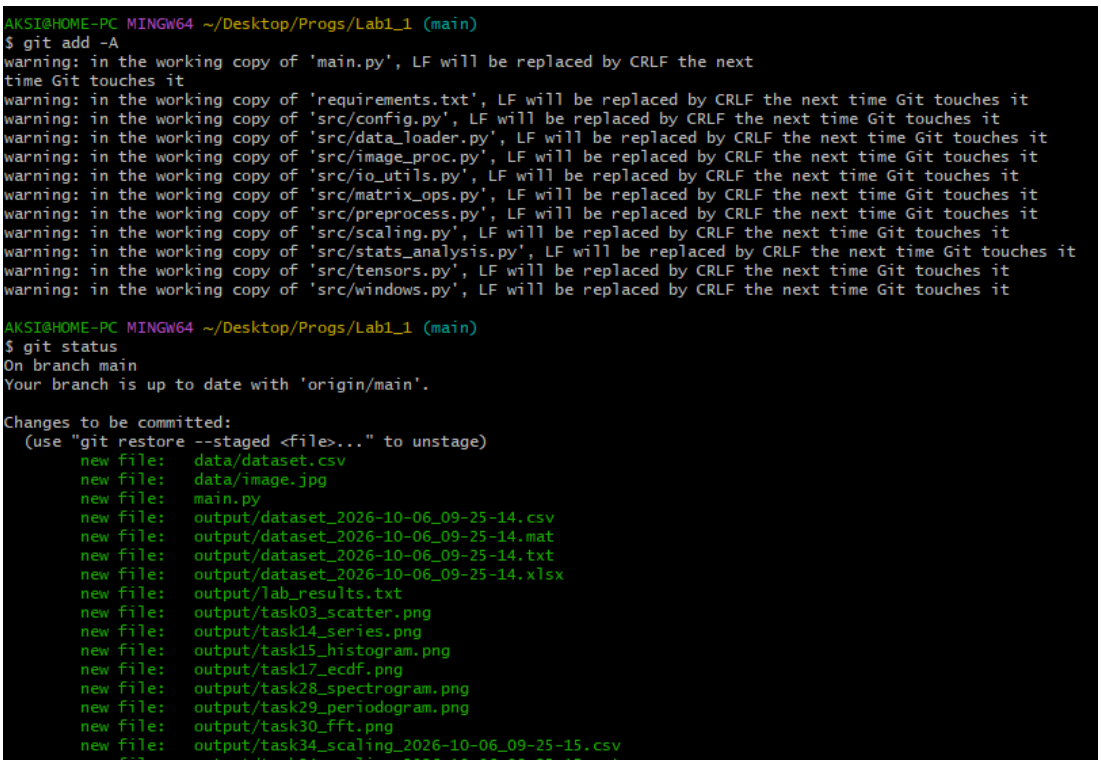


Рисунок 63 – Добавление всех файлов в каталог

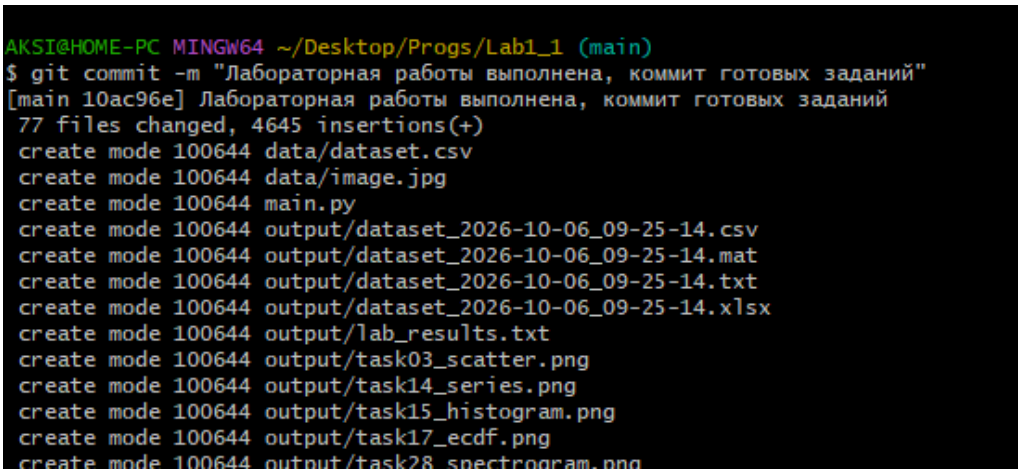


Рисунок 64 – Конечный коммит

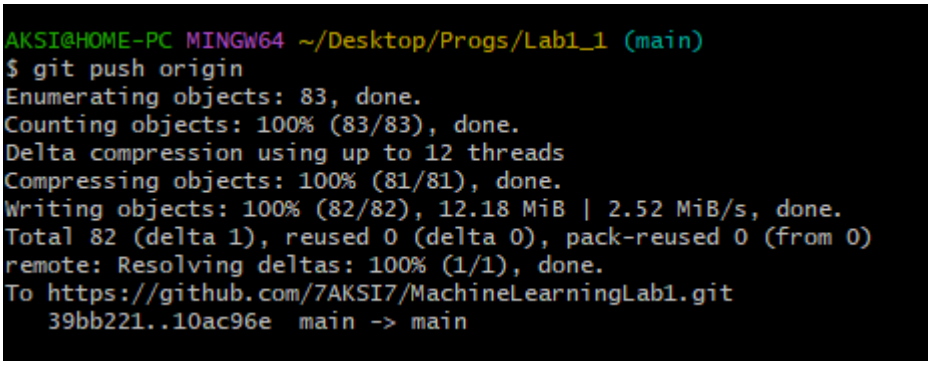


Рисунок 65 – Пуш в репозиторий

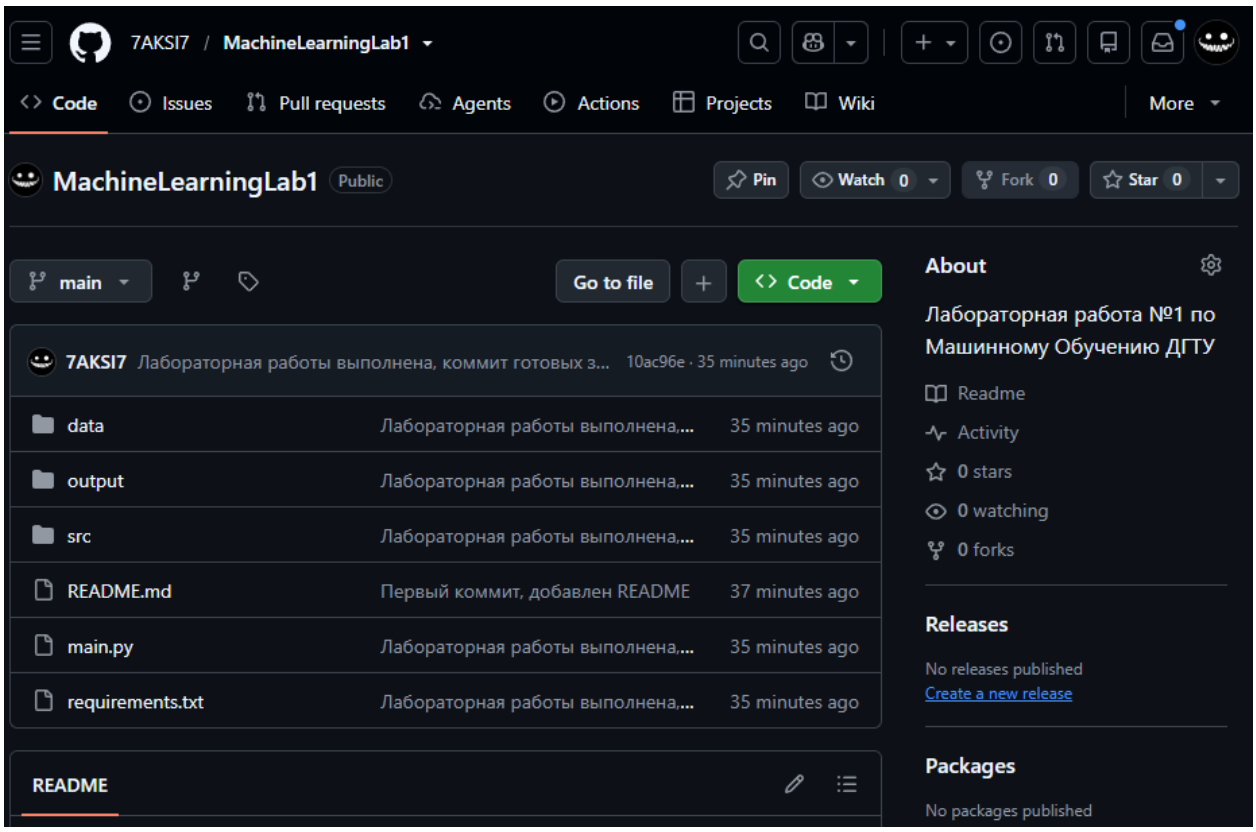


Рисунок 66 – Готовый репозиторий с лабораторной работой

Вывод

В ходе лабораторной работы были изучены и практически применены основные библиотеки Python для машинного обучения. Освоены чтение и запись табличных данных в четырёх форматах; предобработка данных, включая обработку пропусков, приведение типов, масштабирование и разбиение на выборки; статистический анализ, включая описательные статистики, доверительные интервалы, ковариацию и корреляцию, проверку гипотез о распределении, спектральный анализ и интерполяцию; тензорные вычисления в TensorFlow, PyTorch и Keras; работа с разреженными матрицами, обратной матрицей и PCA-пайплайном; базовая обработка изображений средствами SciPy. Все задания выполнены, результаты сохранены в папку output, код воспроизводим благодаря фиксации seed.